

Platform Architecture — AI Lead Generation & Outreach SaaS

Platform Architecture — AI Lead Generation & Outreach SaaS

Status: Proposed — awaiting sign-off before Phase 1 implementation begins. **First tenant:** Virtelon (configured as data, not hard-coded). **Scope of this document:** sections A–I as requested — architecture, folder structure, schema, core interfaces, roadmap, security, multi-tenancy, deployment, and dev order.

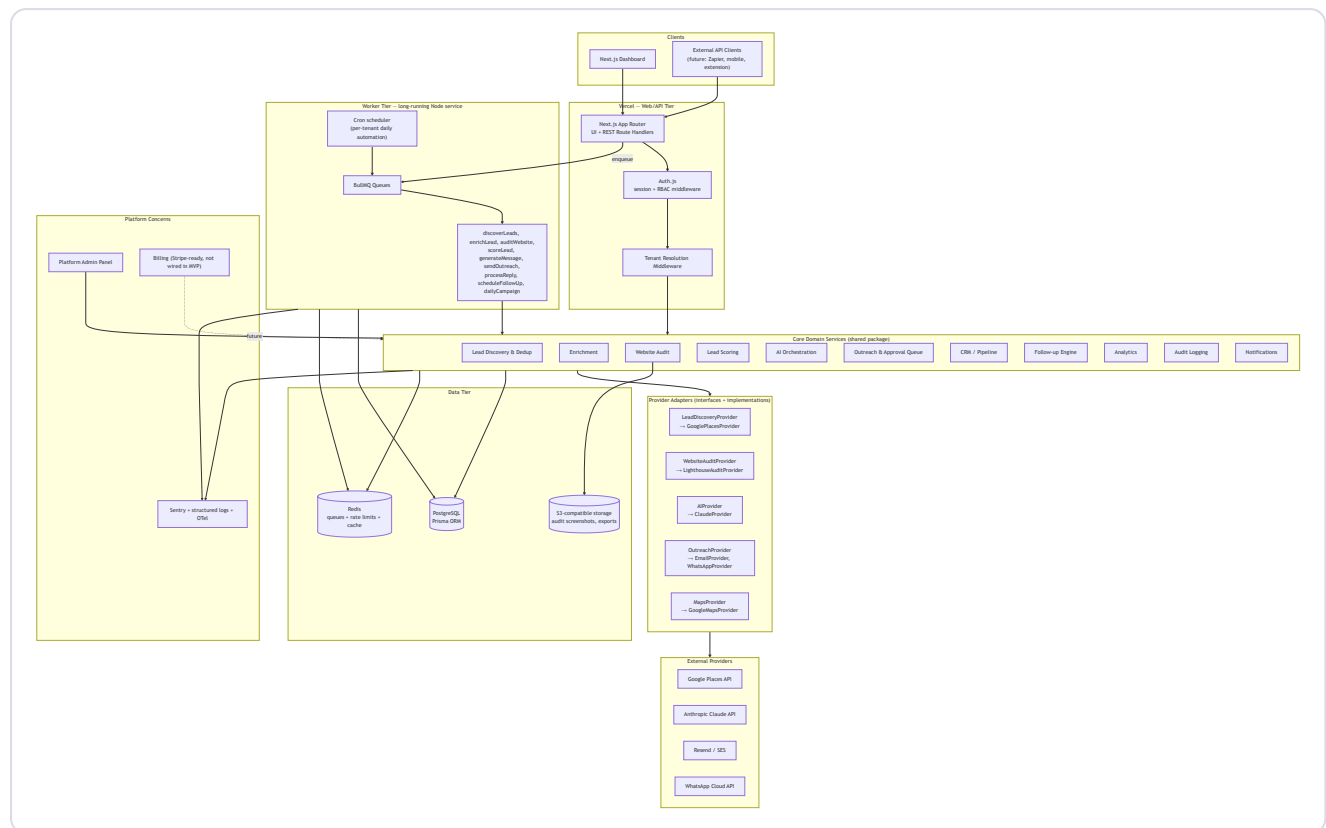
o. Assumptions made (stated, not asked)

A few calls had to be made to keep this moving instead of stalling on questions. All are reversible in code, not in data model shape:

1. **Auth provider:** Auth.js (NextAuth v5) with the Prisma adapter, JWT session strategy carrying `tenantId + role`. Mature, self-hosted (no third-party auth vendor lock-in), supports credentials + OAuth later.
2. **Postgres host:** Neon (serverless Postgres, branch-per-PR previews). Any managed Postgres works — this is a config choice, not an architectural one.
3. **Redis host:** Upstash (serverless-friendly, works from both Vercel functions and a long-running worker).
4. **Object storage:** Cloudflare R2 (S3-compatible, no egress fees — relevant once website-audit screenshots accumulate).
5. **Worker hosting is *not* Vercel.** Explained in §H — this is the one point worth flagging as a deliberate deviation from "Vercel where appropriate."
6. **Lead discovery v1 provider:** Google Places API (Text Search + Place Details) as the first `LeadDiscoveryProvider` implementation — it's an official, ToS-compliant, keyed API with the business attributes this product needs (name, category, rating, reviews, website, phone, address, geo). A second provider isn't built in Phase 2, but the interface is designed so one drops in without touching core logic.
7. **Outreach channel v1:** Email (via Resend or SES) as the first `OutreachProvider` implementation, because it requires no business-verification lead time. WhatsApp Business Platform (Meta Cloud API, official) is designed for from day one but implemented as the second provider once a WhatsApp Business Account is approved — this is an external dependency outside engineering control, not a technical blocker.
8. **AI provider v1:** Claude (Anthropic), as specified.

Everything else below follows directly from your spec.

A. High-Level Architecture



Read of the diagram: the web tier never talks to external providers directly — it always goes through the same core service layer that the worker tier uses. That's the load-bearing decision: *one* set of business logic, invoked synchronously from API routes (for fast, user-facing operations like "regenerate this one message") and asynchronously from workers (for the daily bulk pipeline). No logic duplicated between "the API version" and "the job version."

B. Complete Folder Structure

A **pnpm + Turborepo monorepo**. Reasoning in §H. This is not a single `create-next-app` — it's structured so the worker process and the web process can share the domain layer, database client, and types without copy-pasting.

```

virtelon-platform/
├─ apps/
│   ├─ web/                                # Next.js app (Vercel)
│   │   └─ app/
│   │       ├─ (marketing)/                # public pages (future SaaS marketing site)
│   │       ├─ (auth)/
│   │       │   ├─ login/
│   │       │   ├─ register/
│   │       │   └─ accept-invite/
│   │       └─ (dashboard)/                # tenant-scoped, auth-gated
│   │           ├─ dashboard/
│   │           ├─ leads/
│   │           │   └─ [id]/
│   │           ├─ campaigns/
│   │           │   └─ [id]/
│   │           ├─ inbox/
│   │           ├─ outreach/
│   │           ├─ follow-ups/
│   │           ├─ analytics/
│   │           ├─ settings/
│   │           │   └─ organization/
│   │           │   └─ ai/
│   │           │   └─ lead-sources/
│   │           │   └─ messaging/
│   │           │   └─ scoring-rules/
│   │           │   └─ follow-up-rules/
│   │           │   └─ notifications/
│   │           └─ team/
│   │               └─ integrations/
│   │   └─ admin/                          # platform-admin only, separate authz gate
│   │       ├─ tenants/
│   │       ├─ usage/
│   │       ├─ system-health/
│   │       └─ billing/
│   └─ api/
│       └─ v1/
│           ├─ leads/
│           ├─ campaigns/
│           ├─ outreach/
│           ├─ crm/
│           ├─ analytics/
│           ├─ webhooks/                    # inbound provider webhooks (signed)
│           │   └─ whatsapp/
│           └─ email/

```

```

| | | | | stripe/ # future
| | | | | admin/
| | | | components/
| | | | | ui/ # primitives (button, card, table, badge...)
| | | | | leads/
| | | | | campaigns/
| | | | | outreach/
| | | | | analytics/
| | | | | layout/
| | | | lib/ # web-app-only glue (session helpers, fetchers)
| | | | middleware.ts # tenant resolution + auth + security headers
| | | | | next.config.ts
| | |
| | | | worker/ # long-running Node process (Railway/Fly/Render)
| | | | | src/
| | | | | | queues/ # BullMQ queue + worker definitions
| | | | | | jobs/
| | | | | | | discoverLeads.job.ts
| | | | | | | enrichLead.job.ts
| | | | | | | auditWebsite.job.ts
| | | | | | | scoreLead.job.ts
| | | | | | | generateMessage.job.ts
| | | | | | | queueOutreach.job.ts
| | | | | | | sendOutreach.job.ts
| | | | | | | processReply.job.ts
| | | | | | | scheduleFollowUp.job.ts
| | | | | | | | dailyCampaign.job.ts
| | | | | | | scheduler/ # cron-style per-tenant trigger
| | | | | | | | index.ts # worker bootstrap + health server
| | | | | | | | package.json
| | | |
| | | | packages/
| | | | | db/ # Prisma schema + generated client + tenant-scoped
client wrapper
| | | | | prisma/
| | | | | | schema.prisma
| | | | | | migrations/
| | | | | | src/
| | | | | | | client.ts
| | | | | | | tenant-scope.ts # Prisma Client Extension enforcing tenantId
| | | |
| | | | | core/ # ALL business logic lives here - framework-agnostic
| | | | | | lead-discovery/
| | | | | | | LeadDiscoveryProvider.ts # interface
| | | | | | | providers/

```

```

| | | | └─ google-places.provider.ts
| | | | └─ mock.provider.ts
| | | └─ dedup.ts
| | | └─ normalize.ts
| | └─ lead-enrichment/
| | └─ website-audit/
| | | └─ WebsiteAuditProvider.ts
| | | └─ providers/
| | | | └─ lighthouse.provider.ts
| | | | └─ mock.provider.ts
| | └─ lead-scoring/
| | | └─ scoring-engine.ts
| | | └─ scoring-config.schema.ts
| | └─ ai/
| | | └─ AIProvider.ts
| | | └─ providers/
| | | | └─ claude.provider.ts
| | | | └─ mock.provider.ts
| | | └─ prompts/
| | | └─ schemas/ # zod schemas for AI JSON output
| | └─ outreach/
| | | └─ OutreachProvider.ts
| | | └─ providers/
| | | | └─ email.provider.ts
| | | | └─ whatsapp.provider.ts
| | | | └─ mock.provider.ts
| | | └─ approval-queue.ts
| | | └─ limits.ts # daily/campaign limits, cooldowns
| | | └─ opt-out.ts
| | └─ crm/
| | └─ follow-up/
| | └─ analytics/
| | └─ integrations/
| | | └─ maps/
| | | | └─ MapsProvider.ts
| | | | └─ providers/google-maps.provider.ts
| | └─ billing/ # interfaces only in MVP, no live provider
| | └─ notifications/
| | | └─ NotificationProvider.ts
| | | └─ providers/
| | | | └─ in-app.provider.ts
| | | | └─ email.provider.ts
| | | | └─ slack.provider.ts
| | └─ audit-log/
| | └─ rbac/

```

```

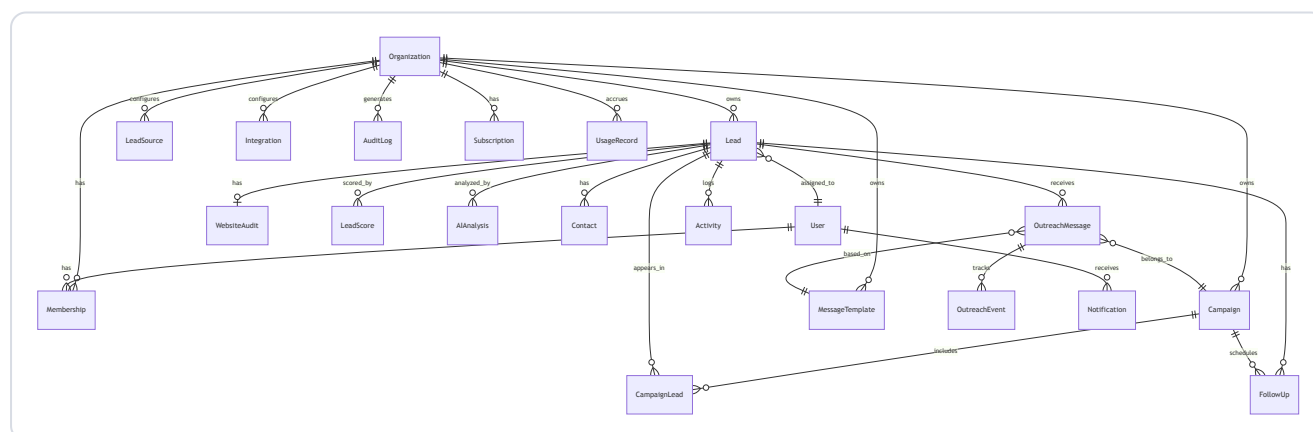
|   |   |   | roles.ts
|   |   |   | permissions.ts
|   |   |
|   |   |   | types/           # shared TS types + zod schemas (Lead, Campaign, etc.)
|   |   |   | config/         # typed env loading (zod-validated), tenant defaults
|   |   |   | ui-tokens/      # design tokens shared by web (and future admin)
|   |   |
|   | docs/
|   |   | ARCHITECTURE.md      # this file
|   |   | api/                 # OpenAPI spec, generated
|   |
|   turbo.json
|   pnpm-workspace.yaml
|   package.json

```

Why this split matters: `packages/core` has zero dependency on Next.js or BullMQ. A job in `apps/worker` and a route handler in `apps/web` both call `packages/core/outreach/approval-queue.ts` — same function, same validation, same audit log write. This is what makes "REST API today, Chrome extension / Zapier / mobile client tomorrow" actually true, and it's what makes the provider-swap requirement (§35) real instead of aspirational.

C. Database Schema / ERD

Core relationships (entities only — see full Prisma draft below for fields):



Prisma schema (design draft — implemented in Phase 1)

```
// packages/db/prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// — Platform / Tenancy —————

model Organization {
  id          String   @id @default(cuid())
  name        String
  slug        String   @unique
  plan        Plan     @default(FREE)
  branding    Json?    // logo url, colors, sender name – tenant config, never hard-coded
  services    Json?    // ["Website Development", "AI Automation", ...] – tenant-defined
  timezone    String   @default("UTC")
  isActive    Boolean  @default(true)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  memberships  Membership[]
  leads        Lead[]
  campaigns    Campaign[]
  leadSources  LeadSource[]
  messageTemplates MessageTemplate[]
  integrations Integration[]
  auditLogs    AuditLog[]
  notifications Notification[]
  subscription Subscription?
  usageRecords UsageRecord[]
  scoringConfig ScoringConfig?
  outreachLimits OutreachLimit?
  followUpRules FollowUpRule[]
  blocklistEntries BlocklistEntry[]

  @@index([slug])
}
```

```

enum Plan {
    FREE
    STARTER
    PROFESSIONAL
    AGENCY
    ENTERPRISE
}

model User {
    id          String    @id @default(cuid())
    email       String    @unique
    name        String?
    passwordHash String?   // null if OAuth-only
    isPlatformAdmin Boolean @default(false)
    createdAt   DateTime @default(now())

    memberships      Membership[]
    assignedLeads     Lead[]        @relation("AssignedLeads")
    activities        Activity[]
    notifications     Notification[]
    auditLogs         AuditLog[]
}

model Membership {
    id          String    @id @default(cuid())
    organizationId String
    userId       String
    role         Role     @default(SALES)
    invitedAt    DateTime @default(now())
    joinedAt     DateTime?

    organization Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
    user           User       @relation(fields: [userId], references: [id], onDelete: Cascade)

    @@unique([organizationId, userId])
    @@index([organizationId])
    @@index([userId])
}

enum Role {
    OWNER
    ADMIN
    MANAGER
    SALES
}

```

```

VIEWER
}

// — Lead Sources & Discovery —————

model LeadSource {
    id                String    @id @default(cuid())
    organizationId    String
    type              String    // "google_places" | "manual_import" | ...
    config             Json      // provider-specific config, secrets referenced not embedded
    isActive           Boolean   @default(true)
    createdAt          DateTime @default(now())

    organization Organization @relation(fields: [organizationId], references: [id], onDelete:
Cascade)
    leads              Lead[]

    @@index([organizationId])
}

model Lead {
    id                String    @id @default(cuid())
    organizationId    String

    businessName      String
    category           String
    subcategory        String?
    phone              String?
    email              String?
    website            String?
    address            String?
    city               String?
    state              String?
    country            String?
    latitude           Float?
    longitude          Float?
    rating             Float?
    reviewCount        Int?
    socialProfiles     Json?    // { instagram, facebook, linkedin, ... }
    businessStatus     String?  // OPERATIONAL | CLOSED_TEMPORARILY | CLOSED_PERMANENTLY | UNKNOWN

    sourceId          String    // LeadSource.id
    externalId         String?   // provider's stable id (e.g. Google Place ID) – dedup
key
    dedupHash          String    // computed fallback hash (name+phone+geo) – dedup key

```

```

    status          LeadStatus @default(NEW)
    websiteStatus    WebsiteStatus @default(UNKNOWN)
    leadScore        Int?
    doNotContact     Boolean @default(false)
    notes            String?

    discoveredAt     DateTime @default(now())
    lastEnrichedAt   DateTime?
    lastContactedAt  DateTime?
    assignedUserId   String?

    organization     Organization @relation(fields: [organizationId], references: [id], onDelete:
Cascade)
    source           LeadSource @relation(fields: [sourceId], references: [id])
    assignedUser     User? @relation("AssignedLeads", fields: [assignedUserId],
references: [id])

    contacts         Contact[]
    websiteAudit      WebsiteAudit?
    scores           LeadScore[]
    aiAnalyses       AIAnalysis[]
    campaignLeads     CampaignLead[]
    outreachMessages OutreachMessage[]
    followUps        FollowUp[]
    activities        Activity[]

    @@unique([organizationId, externalId])
    @@index([organizationId, status])
    @@index([organizationId, dedupHash])
    @@index([organizationId, category, city])
    @@index([organizationId, leadScore])
    @@index([organizationId, discoveredAt])
}

enum LeadStatus {
    NEW
    QUALIFIED
    CONTACTED
    REPLIED
    INTERESTED
    NOT_INTERESTED
    MEETING
    PROPOSAL
    WON

```

```

    LOST
    OPTED_OUT
    DO_NOT_CONTACT
}

enum WebsiteStatus {
    UNKNOWN
    MISSING
    PRESENT
    UNREACHABLE
}

model Contact {
    id          String   @id @default(cuid())
    leadId      String
    name        String?
    role        String?
    email       String?
    phone       String?
    isPrimary   Boolean  @default(false)
    createdAt   DateTime @default(now())

    lead Lead @relation(fields: [leadId], references: [id], onDelete: Cascade)

    @@index([leadId])
}

// — Website Audit & Scoring —————

model WebsiteAudit {
    id          String   @id @default(cuid())
    leadId      String   @unique
    url         String
    reachable   Boolean
    hasHttps    Boolean?
    score       Int?      // 0-100
    performanceHints Json?
    seoFindings Json?
    strengths   String[]
    problems    String[]
    opportunities String[]
    screenshotUrl String? // S3-compatible storage
    rawResult   Json?     // full provider payload for debugging
    auditedAt   DateTime @default(now())
}

```

```

    lead Lead @relation(fields: [leadId], references: [id], onDelete: Cascade)
}

model LeadScore {
  id          String   @id @default(cuid())
  leadId      String
  score       Int
  breakdown   Json      // { websiteOpportunity: 20, businessQuality: 15, ... }
  configVersion Int      // which ScoringConfig version produced this
  computedAt  DateTime @default(now())

  lead Lead @relation(fields: [leadId], references: [id], onDelete: Cascade)

  @@index([leadId])
}

model ScoringConfig {
  id          String   @id @default(cuid())
  organizationId String @unique
  weights     Json      // configurable weight per signal
  threshold   Int       @default(70)
  version     Int       @default(1)
  updatedAt   DateTime @updatedAt

  organization Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
}

model AIAnalysis {
  id          String   @id @default(cuid())
  leadId      String
  provider     String   // "claude"
  model        String
  score        Int?
  priority     String?  // low | medium | high
  reasoning    String[]
  painPoints   String[]
  opportunities String[]
  recommendedService String?
  recommendedPitch String?
  rawResponse  Json
  validated     Boolean @default(false)
  createdAt    DateTime @default(now())

  lead Lead @relation(fields: [leadId], references: [id], onDelete: Cascade)
}

```

```
    @@index([leadId])
}
```

```
// — Campaigns —————
```

```
model Campaign {
  id              String    @id @default(cuid())
  organizationId  String
  name            String
  category        String
  location        String
  service         String
  dailyLeadTarget Int       @default(20)
  minLeadScore   Int       @default(70)
  messageTone     String    @default("professional")
  language        String    @default("en")
  mode            CampaignMode @default(APPROVAL_REQUIRED)
  followUpRuleId  String?
  isActive        Boolean    @default(true)
  scheduleCron    String?    // e.g. "0 9 * * *"
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  organization      Organization @relation(fields: [organizationId], references: [id],
onDelete: Cascade)
  followUpRule      FollowUpRule? @relation(fields: [followUpRuleId], references: [id])
  campaignLeads     CampaignLead[]
  outreachMessages  OutreachMessage[]
  followUps         FollowUp[]

  @@index([organizationId, isActive])
}
```

```
enum CampaignMode {
  MANUAL
  APPROVAL_REQUIRED
  AUTOMATED
}
```

```
model CampaignLead {
  id          String    @id @default(cuid())
  campaignId  String
  leadId      String
  addedAt     DateTime @default(now())
}
```

```

campaign Campaign @relation(fields: [campaignId], references: [id], onDelete: Cascade)
lead      Lead      @relation(fields: [leadId], references: [id], onDelete: Cascade)

@@unique([campaignId, leadId])
@@index([campaignId])
}

// — Outreach —————

model MessageTemplate {
  id            String    @id @default(cuid())
  organizationId String
  name          String
  channel        String    // email | whatsapp
  tone          String?
  language       String?
  bodyTemplate  String?    // optional static fallback / structure hints for AI
  isActive      Boolean    @default(true)
  createdAt     DateTime   @default(now())

  organization Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
  messages      OutreachMessage[]
}

model OutreachMessage {
  id            String    @id @default(cuid())
  leadId        String
  campaignId    String
  templateId    String?
  channel        String
  content        String
  generatedBy    String    // "ai" | "manual"
  status         OutreachStatus @default(DRAFT)
  scheduledFor   DateTime?
  approvedByUserId String?
  approvedAt     DateTime?
  sentAt         DateTime?
  createdAt      DateTime @default(now())

  lead      Lead      @relation(fields: [leadId], references: [id], onDelete: Cascade)
  campaign  Campaign @relation(fields: [campaignId], references: [id], onDelete: Cascade)
  template  MessageTemplate? @relation(fields: [templateId], references: [id])
  events    OutreachEvent[]
}

```

```

    @@index([leadId])
    @@index([campaignId, status])
    @@index([status, scheduledFor])
}

enum OutreachStatus {
    DRAFT
    PENDING_APPROVAL
    QUEUED
    SCHEDULED
    SENDING
    SENT
    DELIVERED
    FAILED
    REPLIED
    INTERESTED
    NOT_INTERESTED
    OPTED_OUT
    FOLLOW_UP_SCHEDULED
    CONVERTED
}

model OutreachEvent {
    id          String   @id @default(cuid())
    messageId   String
    type        String   // queued|sent|delivered|failed|opened|replied|opted_out
    metadata    Json?
    occurredAt  DateTime @default(now())

    message OutreachMessage @relation(fields: [messageId], references: [id], onDelete: Cascade)

    @@index([messageId])
}

model OutreachLimit {
    id              String @id @default(cuid())
    organizationId  String @unique
    dailyLimit      Int    @default(20)
    perCampaignLimit Int?
    cooldownMinutes Int    @default(0)

    organization Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
}

```

```

model BlocklistEntry {
  id          String   @id @default(cuid())
  organizationId String
  contactKey   String   // email or phone, normalized
  reason       String?  // opted_out | manual_block
  createdAt    DateTime @default(now())

  organization Organization @relation(fields: [organizationId], references: [id], onDelete:
Cascade)

  @@unique([organizationId, contactKey])
  @@index([organizationId])
}

// — Follow-ups —————

model FollowUpRule {
  id          String   @id @default(cuid())
  organizationId String
  name        String
  steps       Json      // [{ dayOffset: 3, tone: "friendly" }, { dayOffset: 7, tone: "final"
}]
  stopOnReply Boolean   @default(true)
  stopOnOptOut Boolean   @default(true)
  createdAt    DateTime @default(now())

  organization Organization @relation(fields: [organizationId], references: [id], onDelete:
Cascade)
  campaigns     Campaign[]
}

model FollowUp {
  id          String   @id @default(cuid())
  leadId      String
  campaignId  String
  stepIndex   Int
  scheduledFor DateTime
  status       String   @default("scheduled") // scheduled|sent|cancelled
  createdAt    DateTime @default(now())

  lead      Lead      @relation(fields: [leadId], references: [id], onDelete: Cascade)
  campaign  Campaign @relation(fields: [campaignId], references: [id], onDelete: Cascade)

  @@index([leadId])
}

```

```

    @@index([status, scheduledFor])
}

// — CRM / Activity —————

model Activity {
  id          String   @id @default(cuid())
  leadId      String
  userId      String?
  type        String   // note|status_change|assignment|call|meeting
  content     String?
  metadata    Json?
  createdAt   DateTime @default(now())

  lead Lead @relation(fields: [leadId], references: [id], onDelete: Cascade)
  user User? @relation(fields: [userId], references: [id])

  @@index([leadId])
}

// — Integrations / Billing-ready / Usage —————

model Integration {
  id          String   @id @default(cuid())
  organizationId String
  type        String   // "whatsapp_cloud_api" | "resend" | "google_places" | "stripe"
  status      String   @default("disconnected") // connected|disconnected|error
  encryptedCredentials String? // envelope-encrypted, never plaintext
  metadata    Json?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  organization Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)

  @@unique([organizationId, type])
}

model Subscription {
  id          String   @id @default(cuid())
  organizationId String @unique
  plan        Plan     @default(FREE)
  status      String   @default("active") // active|past_due|canceled
  externalRef String?  // future Stripe subscription id
  currentPeriodEnd DateTime?

```

```
    organization Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
}
```

```
model UsageRecord {
```

```
    id          String    @id @default(cuid())
    organizationId String
    metric       String    // "leads_discovered" | "ai_tokens" | "messages_sent"
    quantity     Int
    periodStart  DateTime
    periodEnd    DateTime
```

```
    organization Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
```

```
    @@index([organizationId, metric, periodStart])
}
```

```
// — Audit & Notifications —————
```

```
model AuditLog {
```

```
    id          String    @id @default(cuid())
    organizationId String
    userId       String?
    action       String    // "campaign.created" | "message.approved" | ...
    entityType   String
    entityId     String
    metadata     Json?
    createdAt    DateTime @default(now())
```

```
    organization Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
```

```
    user        User?      @relation(fields: [userId], references: [id])
```

```
    @@index([organizationId, createdAt])
    @@index([organizationId, entityType, entityId])
}
```

```
model Notification {
```

```
    id          String    @id @default(cuid())
    organizationId String
    userId       String?
    type         String
    title        String
```

```

body          String?
readAt        DateTime?
createdAt     DateTime @default(now())

organization Organization @relation(fields: [organizationId], references: [id], onDelete:
Cascade)
user          User?       @relation(fields: [userId], references: [id])

@@index([organizationId, userId, readAt])
}

```

Indexing notes:

- Every tenant-scoped table leads with an `organizationId` -prefixed composite index matching its dominant query pattern (status filtering, category+city search, score sorting, time-series analytics).
- `Lead` dedup uses **two** keys: `externalId` (provider's stable ID — exact dedup) and `dedupHash` (normalized name+phone+geo fallback — catches leads from different sources describing the same real business). Both are unique/indexed per organization, never globally, so two tenants can legitimately have "the same" business as separate leads.
- `OutreachMessage.status` + `scheduledFor` composite index is what the queue worker scans to find due jobs — this is the hottest query in the system once outreach volume grows.

D. Core Interfaces

These are the seams the entire "no vendor lock-in" requirement depends on. Every one lives in `packages/core/**/*.ts` with zero framework dependency, so they're unit-testable in isolation and swappable without touching route handlers or job definitions.

```
// packages/core/lead-discovery/LeadDiscoveryProvider.ts
```

```
export interface LeadSearchCriteria {
```

```
  category: string;
```

```
  location: string;           // free-text, resolved to geo by the provider
```

```
  minRating?: number;
```

```
  maxRating?: number;
```

```
  requireWebsite?: boolean;
```

```
  limit: number;
```

```
}
```

```
export interface DiscoveredLead {
```

```
  externalId: string;         // provider's stable identifier
```

```
  source: string;             // provider name, e.g. "google_places"
```

```
  businessName: string;
```

```
  category: string;
```

```
  phone?: string;
```

```
  website?: string;
```

```
  address?: string;
```

```
  city?: string;
```

```
  state?: string;
```

```
  country?: string;
```

```
  latitude?: number;
```

```
  longitude?: number;
```

```
  rating?: number;
```

```
  reviewCount?: number;
```

```
  businessStatus?: string;
```

```
  raw: Record<string, unknown>; // original provider payload, retained for debugging/audit
```

```
}
```

```
export interface LeadDiscoveryProvider {
```

```
  readonly name: string;
```

```
  search(criteria: LeadSearchCriteria): Promise<DiscoveredLead[]>;
```

```
}
```

```
// packages/core/website-audit/WebsiteAuditProvider.ts
```

```
export interface WebsiteAuditResult {  
  url: string;  
  reachable: boolean;  
  hasHttps?: boolean;  
  score: number | null;           // 0-100, null if not reachable  
  strengths: string[];  
  problems: string[];  
  opportunities: string[];  
  performanceHints?: Record<string, unknown>;  
  seoFindings?: Record<string, unknown>;  
  screenshotUrl?: string;  
}
```

```
export interface WebsiteAuditProvider {  
  readonly name: string;  
  audit(url: string): Promise<WebsiteAuditResult>;  
}
```

```
// packages/core/ai/AIProvider.ts
```

```
import { z } from "zod";
```

```
export const LeadAnalysisSchema = z.object({  
  score: z.number().min(0).max(100),  
  priority: z.enum(["low", "medium", "high"]),  
  reasoning: z.array(z.string()),  
  painPoints: z.array(z.string()),  
  opportunities: z.array(z.string()),  
  recommendedService: z.string().nullable(),  
  recommendedPitchAngle: z.string().nullable(),  
});
```

```
export type LeadAnalysis = z.infer<typeof LeadAnalysisSchema>;
```

```
export interface OutreachGenerationInput {  
  lead: {  
    businessName: string;  
    category: string;  
    city?: string;  
    website?: string | null;  
    websiteAudited: boolean; // gate: AI may only reference audit facts if this is true  
  };  
  websiteAudit?: WebsiteAuditResult | null;  
  analysis?: LeadAnalysis | null;  
  tenantServices: string[]; // tenant's own service list, not hard-coded  
  tone: "formal" | "friendly" | "concise" | "professional" | "hinglish";  
  language: "en" | "hi" | "hinglish";  
  campaignObjective: string;  
}
```

```
export interface AIProvider {  
  readonly name: string;  
  analyzeLead(input: OutreachGenerationInput): Promise<LeadAnalysis>;  
  generateOutreach(input: OutreachGenerationInput): Promise<string>;  
  generateFollowUp(input: OutreachGenerationInput & { previousMessage: string; stepIndex:  
    number }): Promise<string>;  
  summarizeConversation(messages: string[]): Promise<string>;  
}
```

```
// packages/core/outreach/OutreachProvider.ts
```

```
export interface OutreachSendInput {  
  to: { email?: string; phone?: string };  
  subject?: string;           // email only  
  body: string;  
  metadata?: Record<string, unknown>;  
}  
  
export interface OutreachSendResult {  
  providerMessageId: string;  
  status: "sent" | "queued" | "failed";  
  error?: string;  
}  
  
export interface OutreachProvider {  
  readonly name: string;  
  readonly channel: "email" | "whatsapp" | "sms";  
  send(input: OutreachSendInput): Promise<OutreachSendResult>;  
  /** Verifies inbound webhook signatures before events are trusted. */  
  verifyWebhookSignature(rawBody: string, headers: Record<string, string>): boolean;  
}
```

```
// packages/core/integrations/maps/MapsProvider.ts
```

```
export interface MapsProvider {  
  readonly name: string;  
  geocode(address: string): Promise<{ lat: number; lng: number } | null>;  
  placeDetails(externalId: string): Promise<Record<string, unknown> | null>;  
}
```

```
// packages/core/notifications/NotificationProvider.ts

export interface NotificationPayload {
  organizationId: string;
  userId?: string;
  type: string;
  title: string;
  body?: string;
}

export interface NotificationProvider {
  readonly channel: "in_app" | "email" | "slack" | "whatsapp";
  send(payload: NotificationPayload): Promise<void>;
}
```

Design rule these interfaces enforce: an `AIPProvider.generateOutreach()` implementation is *structurally prevented* from claiming a website was audited when `websiteAudited: false` — that flag is a required input, not something the prompt author has to remember to check. This is the mechanism behind §9's "must not fabricate facts" requirement — enforced in the type, not just the prompt wording.

E. MVP Roadmap

Each phase ends in a working, demoable state — not a stub.

Phase	Deliverable	Exit criteria
1. Foundation	Monorepo, Prisma schema + migrations, Auth.js login/register, Organization+Membership+Role model, tenant-resolution middleware, RBAC guard, empty dashboard shell	A user can register, create an org, invite a teammate with a role, and see a role-gated empty dashboard. Tenant isolation has a passing test suite.
2. Lead Discovery	LeadDiscoveryProvider interface + GooglePlacesProvider + MockProvider, dedup engine, Lead CRUD + list/filter UI	Running a search by category+location returns normalized, deduplicated leads visible in /leads.
3. Audit + Scoring	WebsiteAuditProvider + Lighthouse-based implementation, ScoringConfig + scoring engine, lead score visible on lead detail	Leads with a website get a 0–100 score with strengths/problems/opportunities; leads without one are flagged MISSING and scored accordingly.
4. AI Personalization	AIPProvider + ClaudeProvider, analyzeLead() + generateOutreach(), zod-validated output, AIAalysis persisted	Lead detail page shows AI analysis + a generated draft message, grounded only in verified data.
5. Campaigns + CRM	Campaign CRUD, CampaignLead, pipeline stages, Activity timeline, assignment	A campaign can be created, leads attached, moved through the CRM pipeline, with a visible activity log.
6. Outreach + Approval	OutreachProvider + EmailProvider, approval queue UI (DRAFT → PENDING_APPROVAL → QUEUED), opt-out/blocklist enforcement	A generated message can be reviewed, edited, approved, and sent through a real (or sandbox) email provider; opted-out contacts are provably unreachable.
7. Workers + Scheduling	apps/worker with BullMQ queues for every job in §27, per-tenant cron trigger for the daily pipeline, retry/idempotency tests	The full discover→enrich→audit→score→generate→queue pipeline runs unattended on a schedule, survives a worker crash mid-job, and notifies the user on completion.
8. Analytics	Dashboard metrics + charts (§15), per-campaign/source/category/location breakdowns	Dashboard reflects real counts from the last 30 days with no placeholder numbers.
9. Admin + Billing-ready	Platform admin panel (read-only tenant/usage/system view), Subscription / UsageRecord models wired to real counters, Stripe integration point stubbed but not live	Platform admin can see all tenants' usage; no tenant can see another tenant's data anywhere in the admin surface.
10. Hardening	Rate limiting, security headers, Sentry + structured logging, load-test the queue, full test suite pass, docs	Passes the checklist in §F end-to-end; ready for Virtelon's own daily use at the configured volume.

Phases 2–4 can run with `MockProvider` implementations while real API keys (Google Places, Anthropic) are being provisioned, so nothing blocks on external account approvals except Phase 6's live send (which needs a verified sender domain / WhatsApp Business Account regardless of engineering readiness).

F. Security Model

Concern	Mechanism
Authentication	Auth.js, hashed credentials (argon2) or OAuth, HttpOnly + Secure + SameSite=Lax session cookies
Authorization	RBAC (<code>OWNER/ADMIN/MANAGER/SALES/VIEWER</code>) enforced in a single <code>packages/core/rbac</code> permission-check function, called from every route handler and job — not scattered <code>if</code> statements
Tenant isolation	Prisma Client Extension (<code>tenant-scope.ts</code>) that auto-injects <code>organizationId</code> into every query for tenant-scoped models; raw client access without a resolved tenant context throws at runtime. Tested explicitly (§32).
Input validation	zod schemas at every API boundary (request body, query params) and every AI output boundary
Rate limiting	Upstash Ratelimit, per-IP on public routes (auth), per-org on API routes
CSRF	Same-origin enforcement via Auth.js + <code>SameSite</code> cookies for browser sessions; API-key-authenticated external clients (future) use bearer tokens, which are not CSRF-vulnerable by construction
Secrets	Never in client bundles or <code>.env</code> committed to git. Provider credentials for tenant integrations (<code>Integration.encryptedCredentials</code>) are AES-256-GCM encrypted at the application layer with a key from env/KMS — the DB never holds plaintext
Webhook authenticity	Every inbound webhook (<code>OutreachProvider.verifyWebhookSignature</code>) is signature-verified before its payload is trusted
Transport	HTTPS enforced (HSTS), secure headers middleware (CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy)
SQL injection	Prisma parameterizes everything; no raw SQL string interpolation permitted (lint rule)
XSS	React's default escaping + CSP; any HTML-rendering surface (message previews) is sanitized
Audit trail	Every state-changing action writes an <code>AuditLog</code> row — user, org, action, entity, metadata, timestamp — independent of business-logic success/failure
Data privacy	Lead delete/suppress, do-not-contact enforcement checked before every send, tenant data export endpoint (Phase 9+)

G. Multi-Tenant Strategy

Model: shared database, shared schema, row-level isolation via `organizationId` .

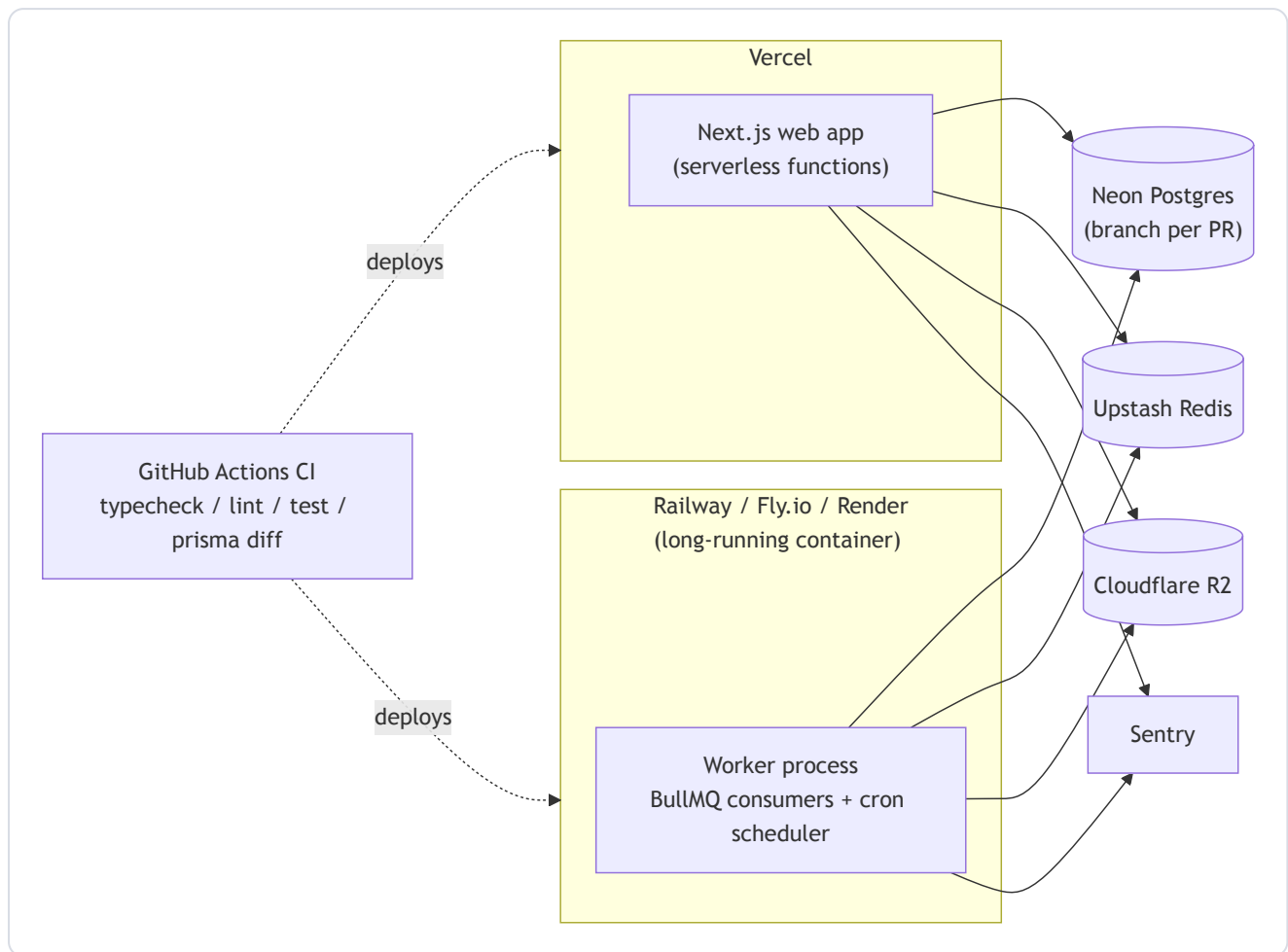
Chosen over schema-per-tenant or database-per-tenant for MVP-to-growth economics — one connection pool, one migration to run, trivial cross-tenant platform-admin queries. The tradeoff (a bug could theoretically leak data across tenants) is closed by making isolation structural rather than convention-based:

1. **Every tenant-scoped Prisma model requires `organizationId` .** No model that stores tenant data is exempt.
2. **A tenant-scoped client wrapper** (`packages/db/src/tenant-scope.ts`) is the *only* sanctioned way core services touch the database. It's constructed from an authenticated session's resolved `organizationId` and transparently filters every read/write. Reaching for the raw Prisma client inside `packages/core` is a code-review red flag, not a style preference.
3. **Platform admin** uses a separate, explicitly-unscoped client, gated by `User.isPlatformAdmin` , used only in `/admin` route handlers — never inside tenant-facing services.
4. **Composite indexes lead with `organizationId` everywhere (§C)**, so isolation doesn't cost query performance.
5. **Tenant isolation has dedicated tests (§32)** that assert org A's session can never read/write org B's rows, including through relations (e.g., can't assign a lead to a user outside the org).

Path to Enterprise isolation: a large enterprise tenant that needs a dedicated database is a config change (point their `organizationId` 's connection string at an isolated DB via a per-tenant datasource resolver), not a rewrite — because the service layer already treats "which DB" as resolved from tenant context rather than a global singleton.

Plan gating (Free/Starter/Professional/Agency/Enterprise): `Organization.plan` + `UsageRecord` + a `packages/core/billing` `PlanLimits` lookup (leads/day, campaigns, users, AI credits, outreach/day) checked at the point of action (e.g., before enqueueing a discovery job). No live billing provider in MVP, but the check is real and enforced from Phase 1 — Virtelon simply runs on a plan record with generous limits.

H. Deployment Architecture



Why the worker isn't on Vercel (the one deliberate deviation from "Vercel where appropriate," flagged per your instruction to explain before making it): Vercel serverless functions are request-triggered, time-boxed, and don't hold persistent connections — BullMQ workers need to `BRPOPLPUSH` /block on Redis continuously and run jobs that can legitimately take minutes (a website audit, a batch of AI calls). That's a long-running-process workload, not a request/response one. So: **web tier on Vercel** (exactly as you specified), **worker tier on a small always-on container** (Railway/Fly/Render — all cheap, simple, git-push-to-deploy, no reason to reach for raw ECS at this stage). Both talk to the same Postgres and Redis. This is the standard pattern for "Next.js on Vercel + real background jobs" — not a sign the stack choice was wrong.

Environments: `dev` (local, docker-compose Postgres+Redis or Neon/Upstash free tier), `preview` (per-PR, Neon database branching gives each PR an isolated DB automatically), `production`.

Config: all secrets via environment variables, validated at boot by a zod schema in `packages/config` — the app refuses to start with a missing/malformed required var rather than failing silently at first use.

I. Recommended Development Order (immediate next steps)

This is Phase 1, broken into the sequence I'd actually execute in:

1. Scaffold the monorepo (`pnpm` , `turborepo` , `apps/web` , `apps/worker` , `packages/db|core|types|config`).
2. Write `schema.prisma` from §C, run the first migration against a local/dev Postgres.
3. Build `packages/config` (env validation) and `packages/db` (client + tenant-scope extension) — these are load-bearing for everything after.
4. Wire Auth.js: register/login, `Organization` creation on first signup, `Membership` with `OWNER` role.
5. Build tenant-resolution middleware + RBAC guard, and write the tenant-isolation test suite *before* any other feature — it's the thing every later phase depends on being trustworthy.
6. Ship the empty dashboard shell + `/settings/organization` + `/team` (invite flow) so Phase 1 is demoable end-to-end.
7. Seed Virtelon as tenant #1 via a seed script (data, not code) to prove the "no hard-coding" claim immediately.

I'd stop here and confirm Phase 1 is solid — types clean, tenant isolation tests green, real login/invite flow working — before starting Phase 2's lead discovery work.

Open items that need your input before Phase 1 code lands

- Confirm the assumed defaults in §0 (Auth.js, Neon, Upstash, R2, Google Places, email-first outreach) or swap any of them — none are hard to change, all are config/vendor choices.
- Virtelon's actual service list and brand colors (for seeding tenant #1 — placeholders otherwise).
- Anthropic API key and Google Places API key availability — Phases 2–4 can proceed against mock providers without them, but real testing needs them eventually.

Ready to start Phase 1 on your go-ahead.